Experiment No. 10
Hyperledger Fabric Network and Asset Management using Chaincode

Aim

To implement a Hyperledger Fabric network and demonstrate asset management using chaincode.

Theory

Hyperledger Fabric is a permissioned and modular blockchain platform for enterprise applications. Unlike public blockchains, participants are known and authorized. A network consists of organizations, peers, ordering services and channels; a **channel** provides a private communication and transaction environment between participating organizations.

Chaincode is the business logic of Fabric (similar to a smart contract). It defines how assets are created, accessed and modified on the ledger. The ledger keeps the current state of assets in the *world state* along with the transaction history. Each asset has an ID, color, size, owner and appraised value. The basic chaincode provides `CreateAsset`, `ReadAsset`, `UpdateAsset`, `TransferAsset`, `DeleteAsset` and `GetAllAssets`.

Transaction flow. A client proposes a transaction, the required peers endorse it, the ordering service orders it into blocks and distributes them, and the peers validate and commit the blocks to their ledgers. This gives controlled access, consistency and transparency while keeping the network permissioned.

Requirements

- Linux machine with Docker and Docker Compose.
- Hyperledger Fabric samples and binaries (Fabric 2.5.16, Fabric CA 1.5.17).
- Go-based `basic` (asset-transfer) chaincode.

Procedure

1. Set up the Hyperledger Fabric test network.
2. Create `mychannel`.
3. Deploy the Go-based `basic` chaincode.
4. Initialize the ledger using `InitLedger`.
5. Read and display the existing assets.
6. Create a new asset `asset10`.
7. Transfer its ownership.
8. Update its properties.
9. Delete the asset and verify the deletion.

Output

```
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Pull complete
6f4ebca3e823: Pull complete
b9649d3f054a: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
===> List out hyperledger images
ghcr.io/hyperledger/fabric-baseos:2.5.16    109b4a15b282        257MB         83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16     807493f095f6       1.01GB          257MB
ghcr.io/hyperledger/fabric-orderer:2.5.16   1d918fc7e1e0        182MB         50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16      58979b65744b        232MB         66.8MB
hyperledger/fabric-baseos:2.5               109b4a15b282        257MB         83.2MB
hyperledger/fabric-baseos:2.5.16            109b4a15b282        257MB         83.2MB
hyperledger/fabric-baseos:latest            109b4a15b282        257MB         83.2MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                807493f095f6       1.01GB          257MB
hyperledger/fabric-ccenv:2.5.16             807493f095f6       1.01GB          257MB
hyperledger/fabric-ccenv:latest             807493f095f6       1.01GB          257MB
hyperledger/fabric-orderer:2.5              1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-orderer:2.5.16           1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-orderer:latest           1d918fc7e1e0        182MB         50.7MB
hyperledger/fabric-peer:2.5                 58979b65744b        232MB         66.8MB
hyperledger/fabric-peer:2.5.16              58979b65744b        232MB         66.8MB
hyperledger/fabric-peer:latest              58979b65744b        232MB         66.8MB
```

Figure 1: Fabric Docker images pulled and listed

```
Using docker and docker-compose
Generating channel genesis block 'mychannel.block'
Using organization 1
/home/chirag/hyperledger/fabric-samples/bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-10-07 10:12:59.658 IST 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-10-07 10:12:59.663 IST 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-10-07 10:12:59.663 IST 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.Options u
nset, setting to tick_interval:"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-10-07 10:12:59.663 IST 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/chirag/hyperledg
er/fabric-samples/test-network/configtx/configtx.yaml
2026-10-07 10:12:59.664 IST 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-10-07 10:12:59.664 IST 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-10-07 10:12:59.666 IST 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
        "name": "mychannel",
        "url": "/participation/v1/channels/mychannel",
        "consensusRelation": "consenter",
        "status": "active",
        "height": 1
}

Channel 'mychannel' created
```

Figure 2: Test network started and channel `mychannel` created

```
ig.json --type common.Config --output /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/modified_config.pb
++ configtxlator compute_update --channel_id mychannel --original /home/chirag/hyperledger/fabric-samples/test-network/channel-ar
tifacts/original_config.pb --updated /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/modified_config.pb --
output /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/config_update.pb --t
ype common.ConfigUpdate --output /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/config_update.json
++ jq .
+++ cat /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}},"data":{"config_update":{' '"channel_id":'
'"mychannel",' '"isolated_data":' '{},' '"read_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '
"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0
"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '""
,' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}' '},'
 '"values":' '{' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mo
d_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"v
alues":' '{},' '"version":' '"0"' '},' '"write_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '
"groups":' '{},' '"mod_policy":' '"Admins",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version"
:' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy"
:' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}
' '},' '"values":' '{' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '[' '{' '"host":' '"peer
0.org2.example.com",' '"port":' 9051 '}' ']' '},' '"version":' '"0"' '},' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"v
ersion":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0
"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '}}}}'
++ configtxlator proto_encode --input /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/config_update_in_env
elope.json --type common.Envelope --output /home/chirag/hyperledger/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.
tx
2026-10-07 10:13:09.448 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-10-07 10:13:09.464 IST 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
hyperledger/fabric-samples/test-network  main
```

Figure 3: Anchor peer set for Org2MSP; channel `mychannel` joined

```
hyperledger/fabric-samples/test-network  main
docker ps                                                                                                                  10:13
CONTAINER ID   IMAGE                               COMMAND             CREATED              STATUS              PORTS
                                                                                                                  NAMES
6ff9dace2c4a   hyperledger/fabric-peer:latest      "peer node start"   About a minute ago   Up About a minute   0.0.0.0:7051->705
1/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp                                                    peer0.org1
.example.com
0eb738ae2486   hyperledger/fabric-peer:latest      "peer node start"   About a minute ago   Up About a minute   0.0.0.0:9051->905
1/tcp, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp                                          peer0.org2
.example.com
c57c8f3f3827   hyperledger/fabric-orderer:latest   "orderer"           About a minute ago   Up About a minute   0.0.0.0:7050->705
0/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp   orderer.ex
ample.com
```

Figure 4: Docker containers running (peer0.org1, peer0.org2, orderer)

```
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/chirag/
hyperledger/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channel
ID mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/chirag/hyperledger/fabric-samples/test-network/
organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCe
rtFiles /home/chirag/hyperledger/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.ex
ample.com-cert.pem --version 1.0 --sequence 1
+ res=0
2026-10-07 10:17:19.864 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [0c142e577cb8b4eba0a417e3d2f9ed7700cb6dfffec63ebd354ae5ea
2ac877ca] committed with status (VALID) at localhost:7051
2026-10-07 10:17:19.871 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [0c142e577cb8b4eba0a417e3d2f9ed7700cb6dfffec63ebd354ae5ea
2ac877ca] committed with status (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
```

Figure 5: `basic` chaincode definition committed on `mychannel`

```
hyperledger/fabric-samples/test-network  main
peer chaincode invoke \                                                                                    10:26
      -o localhost:7050 \
      --ordererTLSHostnameOverride orderer.example.com \
      --tls \
      --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com
-cert.pem" \
      -C mychannel \
      -n basic \
      --peerAddresses localhost:7051 \
      --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
      --peerAddresses localhost:9051 \
      --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
      -c '{"function":"InitLedger","Args":[]}'
2026-10-07 10:27:17.659 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Figure 6: `InitLedger` transaction successful (status:200)

```
hyperledger/fabric-samples/test-network  main
peer chaincode query \
      -C mychannel \
      -n basic \
      -c '{"function":"ReadAsset","Args":["asset1"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5}
```

Figure 7: `ReadAsset` – `asset1` retrieved

```
hyperledger/fabric-samples/test-network  main
peer chaincode query \                                                                                     10:27
      -C mychannel \
      -n basic \
      -c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5},{"AppraisedValue":400,"Color":"red","ID":"asset2",
"Owner":"Brad","Size":5},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10},{"AppraisedValue":600,"
Color":"yellow","ID":"asset4","Owner":"Max","Size":10},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Siz
e":15},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15}]
```

Figure 8: `GetAllAssets` – all existing assets (asset1–asset6)

```
hyperledger/fabric-samples/test-network  main
peer chaincode invoke \                                                                                    10:29
      -o localhost:7050 \
      --ordererTLSHostnameOverride orderer.example.com \
      --tls \
      --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com
-cert.pem" \
      -C mychannel \
      -n basic \
      --peerAddresses localhost:7051 \
      --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
      --peerAddresses localhost:9051 \
      --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
      -c '{"function":"CreateAsset","Args":["asset10","red","10","Chirag","1000"]}'
2026-10-07 10:30:05.428 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Figure 9: `CreateAsset` – `asset10` created

```
hyperledger/fabric-samples/test-network  main
peer chaincode query \
      -C mychannel \
      -n basic \
      -c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":1000,"Color":"red","ID":"asset10","Owner":"Chirag","Size":10}
```

Figure 10: `ReadAsset` – `asset10` displayed (owner Chirag)



Figure 11: `TransferAsset` – ownership of `asset10` transferred to Alice



Figure 12: `UpdateAsset` – `asset10` updated (blue, size 15, value 1500)



Figure 13: `DeleteAsset` – `AssetExists` returns `false` after deletion

Conclusion

The Hyperledger Fabric network was successfully implemented, and the chaincode was used to perform asset creation, retrieval, transfer, update and deletion operations successfully.

Experiment completed successfully.